

Explicação Detalhada do Trabalho Prático de Grafos

Guia didático para entender os blocos do projeto, como eles se conectam e como dividir em 4 semanas

Objetivo deste documento

Explicar o trabalho em linguagem simples, sem partir diretamente para código. A ideia é entender o projeto, os blocos, o fluxo de dados e a divisão por sprints antes da implementação.

Sumário do documento

1. O que é o trabalho em linguagem simples
2. A ideia central: usuários viram vértices e interações viram arestas
3. Os 4 blocos do projeto
4. Bloco 1: Estrutura do Grafo
5. Bloco 2: Minerador do GitHub
6. Bloco 3: Construção dos Grafos
7. Bloco 4: Análise, métricas e exportação para Gephi
8. Como os blocos se conectam no fluxo completo
9. Dica importante para a Semana 2: contrato entre as duplas
10. Divisão por 4 semanas
11. Como evitar que uma dupla trave a outra
12. Perguntas prováveis do professor
13. Checklist mental para saber se o projeto está completo

1. O que é esse trabalho, em linguagem simples

O trabalho pede que o grupo construa uma ferramenta computacional baseada em grafos. Em vez de usar um grafo fictício, o grupo deve usar dados reais de um repositório público do GitHub. A ideia é observar as interações entre pessoas dentro de um projeto de software e representar essas relações em forma de rede.

Na prática, vocês querem responder perguntas como: quem conversa com quem? Quem comenta nas issues de quem? Quem revisa o código de quem? Quem aprova pull requests? Quem faz merge? Quem parece ser mais influente dentro do repositório?

- Escolher um repositório famoso do GitHub, com mais de 5.000 estrelas.
- Buscar dados reais desse repositório usando um minerador.
- Transformar usuários em vértices do grafo.
- Transformar interações em arestas direcionadas.
- Implementar a estrutura de grafo do zero, sem biblioteca pronta de grafos.
- Calcular métricas para analisar a rede de colaboração.
- Exportar o grafo para abrir no Gephi.
- Escrever relatório em LaTeX e apresentar a ferramenta.

Ideia principal

O trabalho não é apenas “pegar dados do GitHub”. O núcleo da nota está em mostrar que vocês sabem modelar o problema como grafo, implementar o grafo do zero, montar a rede de usuários e explicar as análises obtidas.

2. A ideia central: o que é um grafo aqui?

Neste projeto, o grafo representa uma rede de colaboração entre usuários do GitHub. Cada pessoa envolvida no repositório vira um vértice. Cada ação de uma pessoa relacionada a outra pessoa vira uma aresta.

Elemento no GitHub	Elemento no grafo	Exemplo
Usuário do GitHub	Vértice	Alice, Bob, Carol
Comentário em issue ou PR	Aresta direcionada	Alice comentou em uma issue de Bob: Alice -> Bob
Review ou aprovação de PR	Aresta direcionada com peso maior	Carol aprovou PR de Alice: Carol -> Alice
Merge de PR	Aresta direcionada com peso forte	Bob fez merge do PR de Carol: Bob -> Carol
Quantidade/importância da interação	Peso da aresta	Comentário vale menos que merge

2.1 Direção da aresta

A aresta tem direção porque existe alguém que fez a ação e alguém que recebeu aquela ação. Uma regra simples e fácil de explicar é:

quem fez a ação -> dono do item que recebeu a ação

Exemplos:

- Alice comentou na issue criada por Bob: Alice -> Bob.
- Carol revisou o pull request criado por Alice: Carol -> Alice.
- Bob fechou uma issue criada por Carol: Bob -> Carol.
- Alice fez merge de um pull request criado por Bob: Alice -> Bob.

2.2 Peso da aresta

O peso representa a força da relação. Uma interação simples, como comentário, pesa menos. Uma interação mais técnica, como aprovação ou merge, pesa mais.

Tipo de interação	Peso sugerido	Interpretação
Comentário em issue ou pull request	2	Interação leve, discussão ou participação
Abertura de issue comentada por outro usuário	3	Interação um pouco mais forte entre autor e comentador
Revisão ou aprovação de pull request	4	Interação técnica importante
Merge de pull request	5	Interação forte, pois indica integração de código

3. Os 4 blocos do projeto

Para não misturar responsabilidades, o projeto pode ser entendido em 4 blocos. Essa divisão deixa mais fácil implementar, testar e explicar para o professor.

Bloco	Nome	Responsabilidade
Bloco 1	Estrutura do Grafo	Implementar o grafo do zero: classes, métodos, pesos, graus, conectividade e exportação.
Bloco 2	Minerador do GitHub	Buscar dados reais do GitHub e salvar em JSON com cache.
Bloco 3	Construção dos Grafos	Ler JSONs, mapear usuários para índices e montar os 3 grafos separados + grafo integrado.
Bloco 4	Análise e Exportação	Calcular métricas, gerar rankings e exportar para Gephi.

[Bloco 2] Minerador -> salva JSON

[Bloco 3] Construtor -> lê JSON e chama a API do grafo

[Bloco 1] Estrutura do Grafo -> guarda vértices, arestas e pesos

[Bloco 4] Análise/Gephi -> calcula métricas e exporta resultados

4. Bloco 1 — Estrutura do Grafo

O Bloco 1 é o coração do projeto. Antes de minerar qualquer dado do GitHub, vocês precisam ter uma estrutura capaz de armazenar vértices, arestas, pesos e responder perguntas básicas sobre o grafo.

Essa parte precisa ser implementada do zero. Não é permitido usar biblioteca pronta de grafos. A ideia é que o grupo consiga explicar como cada operação funciona.

4.1 Por que existem 3 classes principais?

```
AbstractGraph
|
+-- AdjacencyMatrixGraph
|
+-- AdjacencyListGraph
```

A classe abstrata define o contrato: todo grafo do projeto deve oferecer os mesmos métodos. As duas classes concretas implementam esse contrato de formas diferentes: uma usando matriz de adjacência e outra usando lista de adjacência.

Classe	O que faz	Por que existe
AbstractGraph	Define atributos comuns e métodos que todo grafo deve ter.	Evita duplicação e cria um contrato comum para matriz e lista.
AdjacencyMatrixGraph	Guarda as arestas em uma matriz.	É simples para verificar rapidamente se existe uma aresta $u \rightarrow v$.
AdjacencyListGraph	Guarda as arestas em listas de vizinhos.	É melhor para grafos grandes e esparsos, como redes de usuários.

4.2 Matriz de adjacência

Na matriz, cada posição $[i][j]$ indica se existe aresta do vértice i para o vértice j .

```

      Alice  Bob  Carol
Alice [ 0    1    0 ]
Bob   [ 0    0    1 ]
Carol [ 1    0    0 ]

```

Nesse exemplo, existe Alice $\rightarrow$ Bob, Bob $\rightarrow$ Carol e Carol $\rightarrow$ Alice.

Vantagem	Desvantagem
Verificar se existe aresta é rápido.	Gasta muita memória se houver muitos usuários.
Implementação conceitualmente simples.	Mesmo sem aresta, a matriz precisa reservar espaço.
Boa para explicar em apresentação.	Pode ficar pesada em repositórios muito grandes.

4.3 Lista de adjacência

Na lista, cada vértice guarda apenas os vértices para os quais ele aponta.

```

Alice -> [Bob]
Bob   -> [Carol]
Carol -> [Alice]

```

Vantagem	Desvantagem
Usa menos memória em grafos com poucas conexões.	Verificar uma aresta específica pode exigir percorrer a lista.
Boa para redes grandes de usuários.	A implementação exige mais cuidado com pesos e duplicidade.
Percorrer vizinhos de um usuário é natural.	Pode ser menos direta para quem está aprendendo.

4.4 O que a API do grafo precisa permitir?

Grupo de métodos	Métodos	Para que servem
Básicos	getVertexCount, getEdgeCount, hasEdge, addEdge, removeEdge	Criar, remover e consultar arestas.
Relações	isSucessor, isPredessor, isDivergent, isConvergent, isIncident	Responder perguntas sobre relação entre arestas e vértices.
Graus	getVertexInDegree, getVertexOutDegree	Medir quantas arestas chegam e saem de um vértice.

Pesos	setVertexWeight, getVertexWeight, setEdgeWeight, getEdgeWeight	Guardar importância de vértices e intensidade de arestas.
Grafo todo	isConnected, isEmptyGraph, isCompleteGraph	Analisar características gerais do grafo.
Exportação	exportToGEPHI	Gerar arquivo para visualização no Gephi.

Ponto que o professor pode perguntar

Como se adiciona uma aresta no código? A resposta conceitual é: validar os índices, impedir laço, verificar se a aresta já existe, adicionar apenas se for nova, atualizar contador de arestas e tratar o peso da aresta.

5. Bloco 2 — Minerador do GitHub

O Bloco 2 é responsável por buscar os dados reais do GitHub. Ele conversa com a API do GitHub, coleta informações úteis e salva em arquivos JSON para que o restante do sistema consiga trabalhar sem depender da internet.

5.1 O que o minerador coleta?

- Issues e seus autores.
- Comentários em issues.
- Fechamento de issues e quem fechou.
- Pull requests e seus autores.
- Comentários em pull requests.
- Revisões e aprovações de pull requests.
- Merges de pull requests e quem fez o merge.

5.2 Por que coletar o autor da issue e do PR?

Para criar uma aresta, não basta saber quem comentou ou quem revisou. É preciso saber quem é o dono da issue ou do pull request que recebeu aquela ação.

```
comentador -> autor da issue
revisor    -> autor do pull request
quem mergeou -> autor do pull request
```

5.3 Como ele se encaixa no projeto?

O minerador é a ponte entre o GitHub e o restante do sistema. Ele não monta o grafo diretamente; ele apenas busca os dados, filtra as informações úteis e salva tudo em arquivos JSON. Depois disso, quem lê esses arquivos e monta o grafo é o Construtor de Grafos.

```
GitHub API → Minerador → arquivo JSON → Construtor de Grafos → Grafo
```

Em outras palavras: o minerador coleta a matéria-prima do projeto. O JSON funciona como uma “caixa intermediária” entre a coleta e a construção dos grafos. Isso deixa o sistema mais organizado, mais rápido para apresentar e menos dependente da internet.

Exemplo simples:

```
GitHub API informa: Alice comentou na issue criada por Bob.
Minerador salva no JSON: sourceUser = Alice, targetUser = Bob, type = ISSUE_COMMENT, weight = 2.
Construtor lê o JSON e chama o grafo: addEdge(Alice, Bob) e define peso 2.
```

5.4 Por que salvar em JSON?

A API do GitHub pode ser lenta, pode ter limite de requisições e depende da internet. Durante a apresentação, isso é arriscado. Por isso, o ideal é minerar antes e salvar arquivos locais.

Momento	O que acontece
Antes da apresentação	O minerador busca dados no GitHub e salva JSONs locais.
Durante a apresentação	O sistema lê os JSONs locais, monta o grafo e calcula as métricas rapidamente.

GitHub API -> Minerador -> comentarios.json, fechamentos.json, pull_requests.json

Na apresentação:

JSON local -> Construtor de Grafos -> Grafo -> Métricas

6. Bloco 3 — Construção dos Grafos

O Bloco 3 conecta o mundo do GitHub com o mundo dos grafos. Ele lê os arquivos JSON gerados pelo minerador, transforma usuários em índices numéricos e adiciona arestas nos grafos corretos.

6.1 Por que mapear usuários para números?

As estruturas de matriz e lista de adjacência trabalham melhor com índices inteiros. Por isso, o projeto precisa de um mapeador de usuários, normalmente chamado de UserMapper.

Alice -> vértice 0

Bob -> vértice 1

Carol -> vértice 2

6.2 Os 3 grafos separados

Grafo	O que representa	Exemplo de aresta
Grafo 1	Comentários em issues e pull requests.	Alice comentou em item de Bob: Alice -> Bob
Grafo 2	Fechamento de issue por outro usuário.	Bob fechou issue de Carol: Bob -> Carol
Grafo 3	Revisões, aprovações e merges de PRs.	Carol aprovou PR de Alice: Carol -> Alice

6.3 Grafo integrado

Depois de montar os 3 grafos separados, o projeto monta um grafo integrado. Ele combina todas as interações e soma os pesos quando a mesma relação aparece mais de uma vez.

Alice comentou 3 vezes em itens de Bob: $3 \times \text{peso } 2 = 6$

Alice aprovou 1 PR de Bob: $1 \times \text{peso } 4 = 4$

Peso final Alice -> Bob no grafo integrado = 10

Importante

O grafo integrado é o principal para análise final, pois junta diferentes tipos de colaboração em uma única rede ponderada.

7. Bloco 4 — Análise, métricas e exportação para Gephi

O Bloco 4 usa os grafos prontos para extrair informações. É aqui que vocês transformam o grafo em análise: quem é mais participativo, quem é mais influente, se a rede é densa, quais usuários conectam grupos, entre outros resultados.

7.1 Métricas essenciais

Métrica	O que mostra	Por que é útil
Grau de entrada	Quantas arestas chegam em um usuário.	Mostra quem recebe muitas interações.
Grau de saída	Quantas arestas saem de um usuário.	Mostra quem interage bastante com outros.
Grau total	Soma ou combinação de entrada e saída.	Ajuda a ranquear participação geral.
Densidade	Proporção de conexões existentes na rede.	Mostra se a comunidade é muito ou pouco conectada.
PageRank simples	Influência considerando conexões recebidas.	Ajuda a identificar usuários importantes na rede.

7.2 Métricas avançadas, se der tempo

- Betweenness centrality: identifica usuários que funcionam como ponte entre grupos.
- Closeness centrality: identifica usuários próximos dos demais na rede.
- Coeficiente de agrupamento: mede tendência de formação de grupos fechados.
- Comunidades: identifica grupos que interagem mais entre si.
- Bridging ties: identifica ligações entre comunidades diferentes.
- Assortatividade: observa se usuários muito conectados se ligam a outros muito conectados.

7.3 Exportação para Gephi

O Gephi é uma ferramenta visual para redes. O método `exportToGEPHI` deve gerar um arquivo aceito pelo Gephi, como GEXF ou GraphML. Esse arquivo deve conter vértices, arestas, pesos e rótulos dos usuários.

- O label do nó deve ser o login do usuário, não apenas o número do vértice.
- O peso da aresta deve aparecer no arquivo exportado.
- O grafo integrado deve ser exportado para gerar imagens para o relatório.
- É importante testar a abertura no Gephi antes da apresentação.

8. Como os blocos se conectam no fluxo completo do projeto

Agora que os 4 blocos já foram explicados separadamente, é importante entender como eles trabalham juntos. O projeto completo funciona como uma linha de produção: primeiro os dados são coletados, depois são transformados em grafos, depois o grafo responde consultas e, por fim, as métricas e visualizações são geradas.

Ideia desta seção

Aqui o fluxo será explicado como uma história do começo ao fim. Isso ajuda a entender por que cada bloco existe e por que eles precisam estar separados.

8.1 Passo 1 — Escolha do repositório

O grupo começa escolhendo um repositório público e famoso no GitHub. O enunciado pede um repositório com mais de 5.000 estrelas, porque isso aumenta a chance de existir uma comunidade ativa, com muitas issues, comentários, pull requests, revisões e merges.

Um exemplo de repositório que poderia ser usado é microsoft/vscode. Ele possui muitos usuários, muitas discussões e muitas interações. Essas interações serão a matéria-prima para montar o grafo.

Exemplo de escolha:

Repositório: microsoft/vscode

Motivo: repositório grande, conhecido, com muitas issues, PRs e colaboradores.

8.2 Passo 2 — O minerador busca os dados

Depois de escolher o repositório, o Bloco 2 entra em ação: o Minerador do GitHub. Ele consulta a API do GitHub e faz perguntas como:

- Me dê os comentários das issues desse repositório.
- Me dê os pull requests e seus autores.
- Me dê quem revisou ou aprovou cada pull request.
- Me dê quem fez merge dos pull requests.
- Me dê quem fechou cada issue.

O GitHub responde com dados em formato JSON. Só que esses JSONs podem ser grandes e cheios de informações que não interessam ao trabalho. O papel do minerador é filtrar apenas o que interessa: quem fez a ação, quem recebeu a ação, qual foi o tipo de interação e qual peso será usado depois.

```
GitHub API
-> Minerador
    -> comentarios.json
    -> fechamentos.json
    -> pull_requests.json
```

Por que salvar JSON local?

A mineração deve ser feita antes da apresentação, porque depender da internet e da API do GitHub na hora pode ser arriscado. Na apresentação, o sistema deve conseguir ler os arquivos JSON locais e rodar rápido.

8.3 Passo 3 — O construtor lê os JSONs e monta os grafos

Com os JSONs salvos, o Bloco 3 começa a trabalhar. Ele lê os arquivos gerados pelo minerador e transforma cada usuário em um vértice e cada interação em uma aresta.

A primeira coisa que o construtor precisa fazer é mapear nomes de usuários para números inteiros, porque as estruturas de matriz e lista de adjacência trabalham com índices.

```
Alice -> vértice 0
Bob   -> vértice 1
Carol -> vértice 2
```

Depois disso, para cada interação encontrada no JSON, o construtor adiciona uma aresta no grafo correto.

Dado encontrado no JSON	Grafo usado	Aresta criada
Alice comentou na issue de Bob	Grafo 1 — comentários	Alice -> Bob, peso 2
Bob fechou a issue de Carol	Grafo 2 — fechamento de issue	Bob -> Carol, peso 3 ou regra definida pelo grupo
Carol aprovou o PR de Alice	Grafo 3 — reviews/aprovações/merges	Carol -> Alice, peso 4
Bob fez merge do PR de Carol	Grafo 3 — reviews/aprovações/merges	Bob -> Carol, peso 5

8.4 Passo 3.1 — Montagem do grafo integrado

Depois de montar os 3 grafos separados, o sistema cria o grafo integrado. Esse grafo junta todas as interações em uma única rede ponderada. Se a mesma pessoa interagiu várias vezes com outra, os pesos são somados.

```
Alice -> Bob existe no Grafo 1 com peso 2
Alice -> Bob existe no Grafo 3 com peso 4
```

```
No Grafo Integrado:
Alice -> Bob terá peso 6
```

Esse grafo integrado é muito importante porque representa a rede geral de colaboração do repositório. Ele permite comparar relações fracas, como comentários simples, com relações mais fortes, como reviews e merges.

8.5 Passo 4 — A API do grafo trabalha

Depois que os grafos estão montados, o Bloco 1 é usado para responder perguntas sobre eles. Ou seja, as classes `AbstractGraph`, `AdjacencyMatrixGraph` e `AdjacencyListGraph` deixam de ser apenas estrutura interna e passam a oferecer operações úteis para a análise.

```
grafo.getVertexOutDegree(0)  -> quantas pessoas Alice interage
grafo.getVertexInDegree(1)   -> quantas interações Bob recebe
grafo.isConnected()           -> a rede está toda conectada?
grafo.getEdgeWeight(0, 1)     -> qual a intensidade da relação Alice -> Bob?
```

Essas respostas são usadas pelo Bloco 4 para calcular métricas e gerar conclusões. Por isso, é importante que a API do grafo esteja correta, testada e fácil de explicar.

8.6 Passo 5 — O bloco de análise calcula métricas e exporta

Com o grafo integrado pronto, o Bloco 4 calcula métricas. As métricas transformam o grafo em informações compreensíveis para o relatório e para a apresentação.

Pergunta	Métrica que ajuda a responder
Quem interage mais com outros usuários?	Grau de saída
Quem recebe mais interações?	Grau de entrada
Quem parece ser mais influente?	PageRank ou centralidade
A comunidade é muito conectada?	Densidade
Existem grupos trabalhando juntos?	Comunidades ou clustering
Quem conecta grupos diferentes?	Betweenness ou bridging ties

Exemplo de interpretação de métrica:

```
PageRank rodou:
Alice = 0.35
Bob   = 0.12
```

```
Conclusão possível:
Alice é uma colaboradora mais influente dentro da rede analisada.
```

Depois das métricas, o sistema também exporta o grafo para o Gephi. O Gephi permite visualizar a rede em forma de desenho, com bolinhas e ligações. Essa imagem pode ser usada no relatório e na apresentação.

```
grafo.exportToGEPHI("resultado.gexf")
```

Arquivo gerado: resultado.gexf
Uso: abrir no Gephi e visualizar a rede de colaboradores

8.7 Visão geral em um diagrama

O fluxo completo pode ser resumido assim:

```
GitHub API
  ↓
[Bloco 2] Minerador
  ↓ salva
comentarios.json
fechamentos.json
pull_requests.json
  ↓ lê
[Bloco 3] Construtor de Grafos
  ↓ usa
[Bloco 1] AbstractGraph
      ├── AdjacencyMatrixGraph
      └── AdjacencyListGraph
  ↓ grafos prontos
[Bloco 4] Análise + Exportação
  ↓           ↓
Métricas    resultado.gexf
(no relatório) (abre no Gephi)
```

8.8 O que cada bloco sabe sobre os outros?

Essa parte é muito importante para evitar responsabilidades misturadas. Cada bloco deve saber apenas o necessário para cumprir sua função.

Bloco	O que ele sabe	O que ele não deveria saber
Bloco 1 — Grafo	Sabe sobre vértices, arestas, pesos, graus e operações do grafo.	Não precisa saber que os dados vieram do GitHub.
Bloco 2 — Minerador	Sabe consultar a API do GitHub e salvar JSON.	Não precisa saber como matriz ou lista de adjacência funcionam.
Bloco 3 — Construtor	Sabe ler JSON, mapear usuários e chamar a API do grafo.	Não deve calcular métricas complexas nem consultar diretamente o GitHub.
Bloco 4 — Análise	Sabe receber grafos prontos, calcular métricas e exportar resultados.	Não deve minerar dados nem decidir como o grafo guarda arestas internamente.

Por que isso importa?

Essa separação deixa o projeto mais organizado, facilita testes, evita code smells e ajuda cada membro do grupo a explicar sua parte sem misturar responsabilidades.

8.9 O que acontece na apresentação?

Na apresentação, o grupo deve mostrar o fluxo completo funcionando. O ideal é não depender de internet. Por isso, o minerador pode já ter sido executado antes e os JSONs podem estar salvos localmente.

- Mostrar o repositório escolhido e justificar a escolha.
- Mostrar que o minerador gera ou lê os JSONs locais.
- Mostrar os grafos sendo montados a partir dos JSONs.
- Mostrar algumas operações da API do grafo funcionando.
- Mostrar as métricas calculadas.

- Mostrar o arquivo exportado abrindo no Gephi.
- Mostrar uma imagem da rede de colaboradores.
- Explicar como cada bloco se conecta ao próximo.

8.10 Resumo final do fluxo

Minerar = buscar dados do GitHub.

Construir = transformar JSON em grafos.

Analisar = calcular métricas sobre os grafos.

Exportar = gerar arquivo para visualizar no Gephi.

Apresentar = mostrar o fluxo funcionando e explicar as decisões.

9. Dica importante para a Semana 2: contrato entre as duplas

Uma dica importante é garantir que a Dupla A e a Dupla B não dependam uma da outra de forma desorganizada. Na Semana 2, a Dupla A deve definir claramente como a API do grafo funciona: quais métodos existem, quais são os parâmetros, quais são os retornos e quais exceções podem acontecer.

Com isso, a Dupla B já sabe como o grafo será chamado na Semana 3, mesmo que a implementação ainda não esteja 100% pronta. Isso evita retrabalho e evita que uma dupla fique parada esperando a outra.

Frase-chave

Na Semana 2, a Dupla A entrega o contrato da API do grafo. A Dupla B não precisa esperar o grafo completo para planejar como seus dados serão usados, porque já sabe quais métodos estarão disponíveis.

9.1 O que significa “contrato” na prática?

Contrato é o acordo sobre como uma parte do código será chamada. Por exemplo:

- Como adicionar uma aresta?
- Como definir peso de uma aresta?
- Como recuperar o número de vértices?
- Como consultar se uma aresta existe?
- Como informar que um índice é inválido?
- Como exportar para o Gephi?

9.2 Exemplo conceitual sem código

Minerador gera uma interação:

Alice comentou na issue de Bob, peso 2

Construtor transforma em:

Alice -> Bob

Grafo precisa oferecer operações como:

- adicionar a aresta Alice -> Bob
- definir peso 2 nessa aresta
- consultar depois se essa aresta existe

10. Divisão por 4 semanas

A divisão em 4 semanas ajuda a organizar o projeto como sprints. O ideal é que cada semana tenha uma entrega clara e verificável.

Semana	Foco	Entrega principal
Semana 1	Planejamento e contratos.	Repositório escolhido, modelagem, JSON fictício, relatório iniciado e Etapa 1.
Semana 2	API de grafos e minerador básico.	Grafo funcionando no básico, minerador inicial, testes principais e protótipo.
Semana 3	Integração, grafos finais, métricas e Gephi.	4 grafos montados, métricas essenciais, exportação e roteiro de apresentação.
Semana 4	Finalização.	Testes, relatório, vídeo, apresentação e entrega final.

10.1 Semana 1 — Planejamento e contratos

- Escolher o repositório com mais de 5.000 estrelas.
- Decidir a linguagem: Java ou Python.
- Criar o repositório no GitHub Classroom.
- Definir a divisão de tarefas.
- Criar o diagrama de classes inicial.
- Definir assinaturas dos métodos da API do grafo.
- Definir o formato dos JSONs.
- Criar JSON fictício de interações.
- Iniciar relatório em LaTeX no template da SBC.

10.2 Semana 2 — API de grafos + minerador básico

Dupla	Foco	Atividades
Dupla A	Estrutura do Grafo	Implementar AbstractGraph, matriz, lista, addEdge, removeEdge, hasEdge, pesos, graus e testes.
Dupla B	Minerador básico	Coletar autores, comentários em issues, fechamento de issues, salvar JSON e implementar cache.

10.3 Semana 3 — Integração e análise

Dupla	Foco	Atividades
Dupla A	API restante e construção dos grafos	Finalizar métodos restantes, UserMapper, Grafo 1, Grafo 2, Grafo 3 e Grafo Integrado.
Dupla B	Minerador completo e métricas	Comentários em PRs, reviews, aprovações, merges, JSON final, grau, densidade e PageRank.
Todos	Gephi e apresentação	Exportar arquivo, testar Gephi, gerar imagem, criar roteiro e preparar perguntas.

10.4 Semana 4 — Finalização

- Integrar todos os blocos e testar o fluxo completo.
- Confirmar cobertura de testes próxima ou acima de 90%.
- Revisar code smells e remover duplicações.
- Garantir que nenhuma biblioteca pronta de grafos foi usada.
- Finalizar relatório em LaTeX entre 7 e 15 páginas.
- Gerar PDF final.
- Gravar vídeo de 5 a 10 minutos.
- Preparar apresentação presencial de 10 a 15 minutos.
- Montar ZIP com código fonte, .tex e PDF.
- Entregar no Canvas antes do prazo.

Regra da Semana 4

Na Semana 4, não é recomendado inventar funcionalidades novas. O foco deve ser corrigir, testar, documentar, gravar e ensaiar.

11. Como evitar que uma dupla trave a outra

O maior risco do projeto é uma parte depender demais da outra. Por exemplo, a parte das métricas não pode ficar parada esperando o minerador real ficar pronto. Para evitar isso, o grupo deve usar contratos e dados fictícios desde o início.

Risco	Como evitar
Construtor esperando minerador.	Criar JSON fictício na Semana 1.
Minerador esperando grafo.	Definir contrato da API do grafo na Semana 2.
Apresentação dependendo da internet.	Usar cache e JSONs salvos localmente.
Uma pessoa fazendo tudo.	Dividir por blocos e garantir commits individuais.
Código difícil de explicar.	Separar responsabilidades e evitar classes gigantes.

12. Perguntas prováveis do professor

O professor pode pedir para executar o código e explicar decisões. Algumas perguntas prováveis são:

- Como o método `addEdge` funciona?
- Como vocês impedem laços no grafo?
- Como vocês impedem arestas duplicadas?
- Por que o grafo é direcionado?
- Onde o peso da aresta é armazenado?
- Qual a diferença entre matriz de adjacência e lista de adjacência?
- Por que vocês salvaram os dados do GitHub em JSON?
- Como o minerador transforma uma interação do GitHub em aresta?
- Como o `UserMapper` transforma login de usuário em índice?
- Qual grafo foi usado para calcular as métricas?
- Como vocês garantiram que não usaram biblioteca pronta de grafos?
- Como o arquivo exportado abre no Gephi?
- O que significa um usuário ter alto grau de saída?

- O que significa um usuário ter alto PageRank?
- Como vocês garantiram que o sistema roda rápido na apresentação?

12.1 Resposta conceitual para addEdge

Para adicionar uma aresta $u \rightarrow v$:

1. Verificar se u e v são índices válidos.
2. Verificar se u é diferente de v , pois o grafo não permite laço.
3. Verificar se a aresta já existe.
4. Se não existir, adicionar a aresta.
5. Atualizar o contador de arestas.
6. Definir ou manter o peso da aresta conforme a regra do projeto.

13. Checklist mental para saber se o projeto está completo

Item	Precisa existir?
Repositório público com mais de 5.000 estrelas escolhido.	Sim
Grafo 1 de comentários.	Sim
Grafo 2 de fechamento de issues.	Sim
Grafo 3 de reviews, aprovações e merges.	Sim
Grafo integrado com pesos.	Sim
AbstractGraph.	Sim
AdjacencyMatrixGraph.	Sim
AdjacencyListGraph.	Sim
API obrigatória implementada.	Sim
Sem laços e sem múltiplas arestas.	Sim
Testes unitários.	Sim
Minerador com cache/JSON.	Sim
Métricas essenciais.	Sim
Exportação para Gephi.	Sim
Relatório LaTeX no template SBC.	Sim
PDF do relatório.	Sim
Vídeo de demonstração.	Sim
Apresentação preparada.	Sim
ZIP final com código, .tex e PDF.	Sim

Conclusão

O caminho mais seguro é entender primeiro os 4 blocos, definir contratos cedo, usar JSON fictício para não travar o grupo, salvar dados reais em cache e deixar a Semana 4 apenas para acabamento e apresentação.